Chapter 8 — Introduction to documentation and version control

Chapters 5 through 7 made datasets cleaner, better understood, and fairer to evaluate

Learning objectives

- Connect change tracking to regulated settings such as clinical research

Why documentation enables reproducibility

- Reproducibility requires more than a table of numbers
- Others need the context of collection, processing, and intended use
- Documentation captures metadata, structure, content
- Without that context

Documentation as collaboration

- In multi-person projects, documentation is a shared communication layer
- Clear notes let collaborators grasp structure, content
- Handoffs to new members or external partners become safer when each release explains how

Example 8.1 — Undocumented Image Classification Dataset

- Example 8.1 — hands-on module
- Example 8.1 considers an animal image classification corpus without documentation
- A new user may not know whether labels distinguish dog from puppy
- The lesson is that missing context blocks reuse and invites inconsistent future work
- Explore the chapter example module
- View files: `modules/chapter8/example1/`

Why datasets need version control

- Datasets evolve through cleaning, enrichment
- Avoid overwriting one another’s work
- Tools that combine Git with large-file or data-aware layers help when binaries and shared

Example 8.2 — Feature Drift Across Dataset Versions

- Example 8.2 — hands-on module
- Example 8.2 shows a modeling dataset that gains or modifies features during development
- If a new feature later hurts model quality
- Without that trail, feature drift is hard to diagnose
- Explore the chapter example module
- View files: `modules/chapter8/example2/`

Example 8.3 — Clinical Dataset Change Tracking for Compliance

- Example 8.3 — hands-on module
- Example 8.3 highlights regulated clinical research
- Accountability requires linking each modification to an author and a clear explanation
- Version control therefore supports both scientific reproducibility and compliance evidence
- Explore the chapter example module
- View files: `modules/chapter8/example3/`

Takeaways

- Documentation turns raw observations into interpretable assets
- Collaboration depends on shared context and merge-safe change tracking
- Regulated domains add auditability requirements on top of everyday engineering needs

Next

- Complete the quiz for this part
- The next part moves from why documentation matters to concrete components

Learning objectives

- By the end of this part

Documentation as a user manual

- Dataset documentation functions as a blueprint or user manual
- It describes what the dataset represents, how it was collected or generated
- Reducing misuse in collaborative settings

Core metadata fields

- Metadata is data about data
- Data sources for authenticity and quality assessment
- Version information adds release numbers and change notes so users can track how the

Example 8.5 — Air-Quality Metadata Fields

- Example 8.5 — hands-on module
- Example 8.5 lists metadata for an air-quality monitoring corpus
- Those fields tell a user whether the release fits a modeling or reporting task
- Explore the chapter example module
- View files: `modules/chapter8/example5/`

What a data dictionary records

- A data dictionary explains each variable
- Consistent entries prevent misinterpretation of columns and reduce errors when analysts
- Completeness of the dictionary is as important as completeness of the data rows

Example 8.6 — Age Variable Description

- Example 8.6 — hands-on module
- Example 8.6 shows a concise dictionary description
- Short, unambiguous phrasing beats vague labels that leave units or populations unspecified
- Explore the chapter example module
- View files: `modules/chapter8/example6/`

Example 8.8 — Purchase Amount Data-Dictionary Entry

- Example 8.8 — hands-on module
- Example 8.8 records a structured entry for purchase amount
- Explore the chapter example module
- View files: `modules/chapter8/example8/`

Example 8.8 — listing

```
[
  {
    "Variable Name": "purchase_amount",
    "Data Type": "float",
    "Description": "Total amount spent by the customer in the month",
    "Allowed Values": "Any positive number",
    "Units": "USD"
  }
]
```

Takeaways

- Metadata situates the whole dataset; the data dictionary situates each variable
- Creators, dates, sources, and versions answer who and when
- Structured dictionary entries reduce ambiguity for both humans and automated consumers

Next

- Complete the quiz for this part
- Annotations that capture sampling limits and known faults

Learning objectives

- Write annotation notes that expose sampling limits or known data-quality issues

Codebooks for encoded categories

- A codebook maps numeric or abbreviated codes to full categorical meanings
- Healthcare, government
- Without a codebook, those codes are opaque and analyses risk silent mislabeling of classes

Example 8.9 — Medical Condition Codebook

- Example 8.9 — hands-on module
- Example 8.9 maps disease codes to readable names
- Explore the chapter example module
- View files: `modules/chapter8/example9/`

Example 8.9 — listing

```
Code,Disease Name
1,Hypertension
2,Diabetes
3,Asthma
4,Chronic Obstructive Pulmonary Disease (COPD)
```

Provenance as data lineage

- Also called data lineage, records where data came from and what operations followed
- Links to other datasets that were merged or joined
- Provenance supports integrity checks and explains how validity may have changed

Example 8.10 — Retail Transaction Provenance Trail

- Example 8.10 — hands-on module
- Example 8.10 summarizes a retail trail
- That short narrative lets auditors reconstruct the processing path
- Explore the chapter example module
- View files: `modules/chapter8/example10/`

Annotations and qualitative notes

- Annotations capture context that structured metadata cannot easily hold
- They warn users about sampling bias

Example 8.11 — Urban-Only Sampling Note

- Example 8.11 — hands-on module
- Example 8.11 records a critical sampling limit
- Without that note, downstream users might treat results as nationally generalizable
- Explore the chapter example module
- View files: `modules/chapter8/example11/`

Takeaways

- Codebooks unlock encoded categories
- Together with metadata and dictionaries from the previous part
- Omissions in any layer create silent failure modes

Next

- Complete the quiz for this part
- The next part turns components into practice

Learning objectives

- Recognize a compact multi-section documentation template

A four-move documentation workflow

- Effective documentation is iterative across the dataset lifecycle
- Four moves organize the work

Example 8.13 — Documentation Depth by Intended Use

- Example 8.13 — hands-on module
- Example 8.13 contrasts depth for machine learning training versus exploratory analysis
- Training releases typically need richer preprocessing, label, and split notes
- Purpose decisions drive how much detail each component receives
- Explore the chapter example module
- View files: `modules/chapter8/example13/`

Markdown and everyday formats

- For smaller projects, Markdown or plain text is easy to create, share
- Spreadsheets and templates help standardize metadata tables across many datasets
- Larger programs may add automated metadata generation from tools that already track

Example 8.15 — Markdown Documentation Outline

- Example 8.15 — hands-on module
- Example 8.15 sketches a Markdown layout with overview, table of contents, data dictionary
- Explore the chapter example module
- View files: `modules/chapter8/example15/`

Structured formats and FAIR principles

- Structured metadata such as JSON-LD
- Reusable with sufficient methodology, limitations, and licensing detail

Example 8.19 — Retail Transaction Documentation Template

- Example 8.19 — hands-on module
- Example 8.19 assembles a compact template
- Explore the chapter example module
- View files: `modules/chapter8/example19/`

Common documentation challenges

- Weak organizational buy-in
- Treating documentation as a release gate

Takeaways

- Documentation depth follows intended use
- Maintainable formats range from Markdown to structured Schema.org-style metadata
- FAIR criteria turn completeness into discoverability and reuse
- Templates like the retail example show how overview, dictionary, provenance

Next

- Complete the quiz for this part
- Documentation freezes meaning at a point in time

Learning objectives

- Describe how DVC tracks large datasets stored outside Git

What dataset version control provides

- Dataset version control tracks and manages evolving releases so changes from cleaning
- Collaborating without silent overwrites

Traditional versus automated approaches

- Traditional practice relies on manual names such as versioned filenames or dated folders
- It is simple for tiny projects but lacks granularity, automation, and reliable history
- Scale better for frequent updates and multi-user work, at the cost of a learning curve

Git for small, text-like datasets

- Git is a distributed system designed for source code
- Combined with careful file choices
- Each update becomes a commit with an author and message

Example 8.20 — CSV Dataset Commits in Git

- Example 8.20 — hands-on module
- Example 8.20 stores a CSV dataset in a Git repository and creates a new commit whenever
- That pattern works when files stay modest in size and change alongside analysis code
- Explore the chapter example module
- View files: `modules/chapter8/example20/`

DVC for large remote-backed data

- DVC extends Git-centered workflows for large datasets and models
- Data files live in local or cloud storage while lightweight references remain in the
- Collaborators can pull a specific dataset version so training and analysis use the same

Example 8.21 — DVC Tracking of Cloud-Stored Datasets

- Example 8.21 — hands-on module
- Example 8.21 places a large corpus in cloud object storage while DVC tracks versions
- Analysts keep history and collaboration benefits without stuffing binaries into Git itself
- Explore the chapter example module
- View files: `modules/chapter8/example21/`

Takeaways

- Version control preserves change history for datasets, not only for code
- Manual folder naming does not scale; automated tools do, with some training cost
- Git fits small text-like tables

Next

- Complete the quiz for this part
- Shows when Git LFS is the right layer for large training artifacts

Learning objectives

- Choose among tools using size, collaboration, and metadata needs

Choosing among versioning tools

- Practical choice depends on file size, collaboration model, metadata depth
- Git suits small text-like assets

Example 8.22 — Quilt for Genomic Dataset Versions

- Example 8.22 — hands-on module
- Select the correct release for each analysis
- Package-style sharing helps distributed collaborators avoid informal file copies
- Explore the chapter example module
- View files: `modules/chapter8/example22/`

Example 8.23 — DataHub for Real-Time Feed Versions

- Example 8.23 — hands-on module
- Example 8.23 shows a data engineering team using DataHub to track evolving real-time feeds
- Catalog and lineage features matter when many producers and consumers share enterprise
- Explore the chapter example module
- View files: `modules/chapter8/example23/`

Git LFS for large binaries beside Git

- Git Large File Storage stores heavy binaries outside the main repository while keeping
- It is a strong fit when models or datasets must version with code but would bloat an
- It tracks files rather than full data pipelines or rich dataset catalogs

Example 8.24 — Git LFS for Large Training Artifacts

- Example 8.24 — hands-on module
- Example 8.24 applies Git LFS when training datasets and model files exceed ordinary Git
- Pointers stay in Git while content lives in LFS storage
- Explore the chapter example module
- View files: `modules/chapter8/example24/`

Matching tool to job

- Prefer Quilt when teams need browsable package catalogs
- Many organizations combine tools rather than forcing one system to do every job

Takeaways

- Tool choice follows workload shape
- Quilt and DataHub strengthen sharing and discovery
- Misalignment

Next

- Complete the quiz for this part
- The next part makes tool choice operational

Learning objectives

- Outline a live DVC workflow from init through pull for collaborators

DVC inside machine learning workflows

- Encodes transformation and training steps so experiments remain reproducible
- Teams share references rather than duplicating large files

Example 8.25 — DVC in an ML Pipeline

- Example 8.25 — hands-on module
- Storage burden stays on remotes
- Explore the chapter example module
- View files: `modules/chapter8/example25/`

Pipeline stages as executable history

- A DVC pipeline declares stages with commands, dependencies
- When inputs or code change
- That dependency graph is the operational heart of reproducibility

Example 8.28 — DVC Pipeline Stages for Training

- Example 8.28 — hands-on module
- Example 8.28 defines preprocessing, training
- Explore the chapter example module
- View files: `modules/chapter8/example28/`

Example 8.28 — listing

```
stages:
  preprocessing:
    cmd: python preprocess.py data/raw_data.csv
data/preprocessed_data.csv
    deps:
      - data/raw_data.csv
    outs:
      - data/preprocessed_data.csv
  training:
    cmd: python train_model.py data/preprocessed_data.csv
models/model.pkl
    deps:
      - data/preprocessed_data.csv
    outs:
      - models/model.pkl
  evaluating:
    cmd: python evaluate_model.py models/model.pkl
```

Continuous integration for dataset updates

- Trigger training when code or data lands on a protected branch
- Automation reduces forgotten manual steps and keeps shared remotes current for the team’s

Example 8.29 — Live Demonstration of DVC Automation

- Example 8.29 — hands-on module
- Example 8.29 outlines a live walkthrough
- Explore the chapter example module
- View files: `modules/chapter8/example29/`

Example 8.29 — listing

```
git init
dvc init
dvc remote add -d myremote s3://mybucket/datasets
```

Takeaways

- DVC pipelines turn versioned data into reproducible workflows
- Stage YAML records commands, dependencies, and outputs
- Together they close the gap between “we have a dataset” and “we can retrain from the same

Next

- Complete the quiz for this part
- The next part turns documentation and versioning into day-to-day operating practice

Learning objectives

- Choose Git LFS or DVC when plain Git cannot hold large binaries

Documentation as a living checklist

- Every release should carry the same minimum set
- Update the checklist whenever the data change rather than rewriting definitions from
- Consistency across releases matters more than occasional long essays

Example 8.30 — Codebook for Categorical Labels

- Example 8.30 — hands-on module
- Example 8.30 reminds teams to keep a codebook for categorical encodings
- Explore the chapter example module
- View files: `modules/chapter8/example30/`

Example 8.30 — listing

```
{
  "age": {
    "type": "integer",
    "description": "Age of the individual in years",
    "range": "0-120",
    "unit": "years"
  },
  "gender": {
    "type": "categorical",
    "description": "Gender of the individual",
    "values": ["male", "female", "non-binary"]
  }
}
```

Provenance and version-control habits

- Provenance should record sources, instruments, cleaning scripts, transforms, joins
- Dataset changes should be committed as logical units
- Git suits scripts and small text artifacts

Example 8.33 — Descriptive Dataset Commit Message

- Example 8.33 — hands-on module
- Example 8.33 shows a useful commit message
- Explore the chapter example module
- View files: `modules/chapter8/example33/`

Naming and collaborative practice

- Clear names encode project, version, date
- Separate raw, processed, intermediate, model, and output folders
- Use branches or separate experimental versions, sync regularly

Example 8.34 — Large Binary Files Beyond Plain Git

- Example 8.34 — hands-on module
- Example 8.34 explains why plain Git is inefficient for large binaries and recommends Git
- That pattern keeps repositories performant
- Explore the chapter example module
- View files: `modules/chapter8/example34/`

Takeaways

- Best practice is operational consistency
- Collaboration and compliance rest on review, sync habits

Next

- Complete the quiz for this part
- The next part shows these practices in domain settings

Learning objectives

- Outline governed package sharing for clinical collaboration with role-based access

Research, industry, and clinical stakes

- Research reuse fails when undocumented transforms hide in informal notes
- Industrial retraining fails when two runs use different sensor snapshots under the same
- Clinical collaboration fails when sensitive releases lack auditability and access control
- The case studies map practices to those failure modes

Example 8.38 — Climate Agriculture Research Case Study

- Example 8.38 — hands-on module
- Socioeconomic indicators across teams
- One metadata template, one dictionary
- Explore the chapter example module
- View files: `modules/chapter8/example38/`

Example 8.38 — listing

```
{
  "precipitation_mm": {
    "description": "Monthly precipitation in millimeters",
    "source": "Satellite-derived weather product",
    "unit": "mm",
    "method": "Interpolated before regional aggregation"
  }
}
```

Example 8.39 — Predictive Maintenance Versioning Case Study

- Example 8.39 — hands-on module
- Example 8.39 centers on daily factory sensor feeds and failure-prediction retraining
- DVC tracks each raw batch, preprocessing output
- Explore the chapter example module
- View files: `modules/chapter8/example39/`

Example 8.39 — listing

```
dvc add sensor_data/April_2024.csv
git commit -m "Added April 2024 sensor data"
dvc push
```

Example 8.40 — Clinical Trial Collaboration Case Study

- Example 8.40 — hands-on module
- Example 8.40 involves clinical data managers, statisticians, regulators
- A governed catalog, release notes, role-restricted access
- Explore the chapter example module
- View files: `modules/chapter8/example40/`

Example 8.40 — listing

```
quilt push my_dataset_v1
quilt push my_dataset_v2
```

Cross-case lessons

- Shared templates and provenance scale multi-collector research
- Pinning data to models stabilizes industrial retraining
- Governed catalogs with audit logs stabilize regulated collaboration
- Across domains

Takeaways

- Climate research needs coherent multi-source documentation
- The same chapter principles adapt to each constraint without requiring identical tooling

Next

- Complete the quiz for this part
- Emerging ideas such as auditability mechanisms and edge-sensor version sync

Learning objectives

- Outline emerging trends including auditability ledgers and real-time or edge versioning

Automating documentation updates

- Manual documentation drifts when datasets update frequently or many teams contribute
- Integrating metadata extraction into processing pipelines reduces error and lag
- Tools that already version data can emit creation dates, sources, transforms

Example 8.41 — Auto-Generated Training Metadata

- Example 8.41 — hands-on module
- Example 8.41 shows JSON metadata emitted after model training
- Explore the chapter example module
- View files: `modules/chapter8/example41/`

Example 8.41 — listing

```
{
  "dataset_version": "v1.0",
  "creator": "Data Science Team",
  "creation_date": "2024-12-01",
  "last_modified": "2024-12-02",
  "transformation": "Missing values imputed using median",
  "source": "Company internal database",
  "data_source": "https://company.internal/data_source_v1"
}
```

Pipelines, lineage, and compliance

- Version control should travel with pipeline stages from ingestion through cleaning to
- Lineage platforms help demonstrate where data came from, how they were transformed
- Sensitive assets may further require encryption and restricted remotes alongside ordinary

Example 8.43 — DVC Stages in a Sensor Pipeline

- Example 8.43 — hands-on module
- Example 8.43 wires ingest, clean
- Explore the chapter example module
- View files: `modules/chapter8/example43/`

Example 8.43 — listing

```
version: "v1.0"
stages:
  - name: ingest_data
    cmd: python ingest_data.py
    deps:
      - sensor_data/raw_data.csv
  - name: clean_data
    cmd: python clean_data.py
    deps:
      - sensor_data/raw_data.csv
  - name: train_model
    cmd: python train_model.py
    deps:
      - cleaned_data.csv
```

Emerging trends

- Version sync across cloud and edge devices
- Not every idea is production-ready

Example 8.47 — Blockchain Ideas for Version Auditability

- Example 8.47 — hands-on module
- Aiming for verifiable authenticity and integrity among stakeholders
- Explore the chapter example module
- View files: `modules/chapter8/example47/`

Takeaways

- Advanced practice automates metadata, embeds versioning in pipelines
- Emerging ideas push toward continuous, distributed, and highly auditable histories
- The durable lesson of the chapter remains

Next

- Complete the quiz for this part
- Complete the quiz for this part